STOCHASTIC MODELING

MODULE 5 | LESSON 2

MARKOV CHAINS: AN APPLICATION

Reading Time	50 minutes
Prior Knowledge	Markov models, Monte Carlo

|**Keywords** |Markov Process, Markov chains, Transition Matrix, Bond valuation

---

As we become familiar with Markov models, it is convenient to see them in practice. In this lesson, we will use Markov chains to model firm risk, as proxied by its credit rating. Then, we will see how we can use this information for bond valuation.

In [1]:
```
from IPython.display import VimeoVideo

# Bigger video
VimeoVideo("1121456979", h="3298dbabb7", width=700, height=450)
```

Out[1]:

1. Markov chains and Credit Ratings

Credit ratings provide investors with valuable information to assess firms' abilities to meet their debt obligations, i.e., the probability of default.

Ratings are published periodically and change from time to time to reflect firms' changes in credit risk. Thus, we can use a Markov chain model to identify the dynamics of a firm's credit rating.

To define the problem formally, consider a homogeneous Markov chain S_t defined on the state space $\mathcal{S}=\{1,2,...,D-1,D\}$ with transition matrix P. State 1 represents the highest credit class, state 2 the second highest, and so on, while state $D-1$ refers to the lowest credit class, and state D represents default (an absorbing state).

The table below shows the actual transition probabilities for credit ratings published by Standard & Poor's (NR stands for *'rating withdrawn'*). Obviously, this is the key ingredient for our model, but also the type of information that is harder to get in practice. For now, let's leverage the data from S&P:

From/to	AAA	AA	A	BBB	BB	B	CCC/C	D	NR
AAA	87.06	9.06	0.53	0.05	0.11	0.03	0.05	0	3.11
AA	0.48	87.23	7.77	0.47	0.05	0.06	0.02	0.02	3.89
A	0.03	1.6	88.58	5	0.26	0.11	0.02	0.05	4.35
BBB	0	0.09	3.25	86.49	3.56	0.43	0.1	0.16	5.92
BB	0.01	0.03	0.11	4.55	77.82	6.8	0.55	0.63	9.51
B	0	0.02	0.07	0.15	4.54	74.6	4.96	3.34	12.33
CCC/C	0	0	0.1	0.17	0.55	12.47	43.11	28.3	15.31

Let's start by importing libraries needed and setting up some printout options:

In [2]:
```
# Import libraries
import matplotlib.pyplot as plt
import numpy as np
from numpy.random import rand, seed
from scipy.stats import norm

# Printout options and fixing random seed
seed(12345)
np.set_printoptions(precision=3, suppress=True)
```

Then, let's hardcode the previous transition matrix. In the process, we will get rid of the "NR" type, as its not really clear what rating withdrawn can mean in this context. Also, this requires us to further normalize the transition probabilities so that they sum up to one again (i.e., after removing NR):

In [3]:
```
P0 = np.array(
    [
        [87.06, 9.06, 0.53, 0.05, 0.11, 0.03, 0.05, 0.0, 3.11],
```

```
        [0.48, 87.23, 7.77, 0.47, 0.05, 0.06, 0.02, 0.02, 3.89],
        [0.03, 1.6, 88.58, 5.0, 0.26, 0.11, 0.02, 0.05, 4.35],
        [0, 0.09, 3.25, 86.49, 3.56, 0.43, 0.1, 0.16, 5.92],
        [0.01, 0.03, 0.11, 4.55, 77.82, 6.8, 0.55, 0.63, 9.51],
        [0.0, 0.02, 0.07, 0.15, 4.54, 74.6, 4.96, 3.34, 12.33],
        [0.0, 0.0, 0.1, 0.17, 0.55, 12.47, 43.11, 28.3, 15.31],
        [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 100.0, 0.0],
    ])

# Ignore the NR type
P = P0[:, 0 : P0.shape[1] - 1]

# Normalize the transition probabilities
P = (P.T / np.sum(P, axis=1)).T

print(P)
```

```
[[0.899 0.094 0.005 0.001 0.001 0.    0.001 0.    ]
 [0.005 0.908 0.081 0.005 0.001 0.001 0.    0.    ]
 [0.    0.017 0.926 0.052 0.003 0.001 0.    0.001]
 [0.    0.001 0.035 0.919 0.038 0.005 0.001 0.002]
 [0.    0.    0.001 0.05  0.86  0.075 0.006 0.007]
 [0.    0.    0.001 0.002 0.052 0.851 0.057 0.038]
 [0.    0.    0.001 0.002 0.006 0.147 0.509 0.334]
 [0.    0.    0.    0.    0.    0.    0.    1.   ]]
```

Then, it's time to check the convergence properties of the transition matrix. The current matrix P gives us the short-term (e.g., 1-year) transition probabilities, but what about the long-term behavior of these probabilities?

As we saw in the previous lesson, we can check this by simply using powers of the transition matrix. This is important to assess for a series of reaons:

1. **Steady-state distribution**: informs about long-run properties of firms in each tranche of the P matrix. For instance, computing P^{10} we can assess the distribution of firms in each rating within 10 years.

2. **Absorbing state**: what's the probability for each firm in the long-run to eventually enter into default state? This is something we can assess using, for instance, P^{200}.

3. **Long-run risk management**: if we want to price a long-term bond, we might be interested in transition probabilities in the long run, rather than on a short time span.

In [4]:
```
# Verify where the ratings process converges given these estimates

P10 = np.linalg.matrix_power(P, 10)
print('P^{10}', P10)

P50 = np.linalg.matrix_power(P, 50)
print('P^{50}', P50)

P200 = np.linalg.matrix_power(P, 200)
print('P^{200}', P200)
```

```
P^{10} [[0.353 0.387 0.192 0.046 0.011 0.006 0.001 0.005]
 [0.021 0.42  0.395 0.124 0.021 0.01  0.002 0.008]
 [0.003 0.083 0.541 0.276 0.055 0.023 0.003 0.016]
 [0.001 0.019 0.182 0.515 0.156 0.07  0.01  0.047]
 [0.    0.005 0.044 0.203 0.311 0.228 0.031 0.178]
 [0.    0.002 0.012 0.054 0.154 0.302 0.045 0.432]
 [0.    0.001 0.005 0.018 0.052 0.115 0.018 0.79 ]
 [0.    0.    0.    0.    0.    0.    0.    1.   ]]
P^{50} [[0.013 0.104 0.293 0.262 0.096 0.062 0.009 0.161]
 [0.006 0.075 0.26  0.263 0.103 0.071 0.01  0.211]
 [0.004 0.056 0.221 0.247 0.105 0.075 0.011 0.282]
 [0.003 0.038 0.163 0.204 0.096 0.073 0.01  0.412]
 [0.001 0.019 0.088 0.122 0.065 0.054 0.008 0.642]
 [0.001 0.009 0.041 0.06  0.035 0.031 0.004 0.819]
 [0.    0.003 0.015 0.022 0.013 0.011 0.002 0.934]
 [0.    0.    0.    0.    0.    0.    0.    1.   ]]
P^{200} [[0.001 0.011 0.044 0.053 0.024 0.018 0.003 0.847]
 [0.001 0.01  0.04  0.047 0.022 0.016 0.002 0.862]
 [0.001 0.009 0.035 0.042 0.019 0.014 0.002 0.878]
 [0.    0.007 0.028 0.033 0.015 0.011 0.002 0.905]
 [0.    0.004 0.016 0.019 0.009 0.007 0.001 0.945]
 [0.    0.002 0.008 0.009 0.004 0.003 0.    0.973]
 [0.    0.001 0.003 0.003 0.002 0.001 0.    0.99 ]
 [0.    0.    0.    0.    0.    0.    0.    1.   ]]
```

With this information, we can actually simulate, based on the 1-year transition probabilities, how long can it take for a firm to default, which has important implications down the road:

In [5]:
```
# Model ratings from string to numeric using a dictionary

RATINGS = dict(
    [("AAA", 0),
        ("AA", 1),
        ("A", 2),
        ("BBB", 3),
        ("BB", 4),
        ("B", 5),
        ("CCC", 6),
        ("D", 7),])
```

Suppose a firm has a current rating 'CCC', how long does it take for it to default?

In [6]:
```
CURR_RATING = "CCC"    # Set current rating

N_HISTORIES = 1000    # Number of simulations in our MonteCarlo simulation
LEN_HIST = 100         # We are assessing this over a maximum of 100 years

histories = np.zeros((N_HISTORIES, LEN_HIST), np.int8)  # Initialize the matrix
histories[:, 0] = RATINGS[CURR_RATING]    # Departing from current rating
randarray = rand(N_HISTORIES, LEN_HIST)    # Random sampling array
default_time = np.zeros(N_HISTORIES)       # Initialize default time vector
default_sum = 0                           # We depart from 0 defaults by the firm
```

We can simulate for N_histories over 100 years the potential defaults the firm would incurr in. Then, we can average over N_histories to obtain an estimate of expected time-to-default:

In [7]:
```
# Simulate firm next-years credit rating based on P matrix:
for i in range(0, N_HISTORIES):
    for j in range(1, LEN_HIST):
        for r in RATINGS:
            if randarray[i, j] < np.cumsum(P[histories[i, j - 1], :])[RATINGS[r]
                histories[i, j] = RATINGS[r]
                break
        if histories[i, j] == RATINGS["D"]:   # simulation stops once the firms
            break
    # Compute the average time to default
    if np.max(histories[i, :]) == RATINGS["D"]:
        where_default = np.where((histories[i, :] == RATINGS["D"]))
        default_time[i] = where_default[0][0]
        default_sum += 1
    else:
        default_time[i] = 0.0
```

In [8]:
```
print("Default time:", np.sum(default_time) / default_sum)
```

```
Default time: 8.28688524590164
```

So, under these transition probabilities and the simulation we have performed, on average a firm starting with a rating of "CCC" today would default in a bit more than 8 years

2. Bond Valuation with ratings' transition matrices

As you can guess, the kind of estimations we performed in the previous section can be very useful when we are pricing assets that intrinsically depend on the firms' survival, for example corporate bonds. Let's see next an example of how to use Markov chains and transition matrices to price corporate bonds.

Consider a corporation that has an outstanding bond that has been rated as BB and matures in 5 years, with a 4% annual coupon. The current forward interest rates for years 1 to 4 appear in the table below (in percentages).

Category	Year 1	Year 2	Year 3	Year 4
AAA	2.70	3.13	3.55	3.84
AA	2.74	3.17	3.59	3.88
A	2.79	3.24	3.70	3.99
BBB	3.08	3.50	3.94	4.22
BB	4.16	4.52	5.09	5.45
B	4.54	5.27	6.02	6.39
CCC	11.29	11.27	10.52	10.14

The matrix above gives us an estimation of the term structure of interest rates for each rating class. For instance, the 2.7% for AAA in year 1, means that the expected interest rate from year 1 to year 2 is 2.7% for AAA bonds.Thus, we can compute the bond value, in one year, for each possible future state. Notice that we do not need the current spot

rate (i.e., the interest rate from year 0 to year 1) because we are doing a one-year forward valuation (that is, we will get the value of the bond at the end of the first year).

With this information and the transition matrix on credit ratings we have seen in the previous section, we can compute an estimate of the 1-year-ahead value of the bond. In order to do that we need to:

- **Step 1**. Compute the present value of the bond + coupon in one year's time using the forward rates.
- **Step 2**. Compute the expected value of the bond and the distribution of value changes using the transition matrix.

Let's start by considering the information regarding the bond we want to price, such as the current rating, face value, and annual coupon. Additionally, we need to make an estimate of how much of the value of the bond would be recoverable if the firm defaults. In this case, let's assume this **recovery rate** is 62%:

In [9]:
```
CURR_RATING = "BB"
FACE = 100.0
COUPON = 4.0
D_RECOVERY = 62
MAT = 5
```

Then, we are going to code the information on forward rates:

In [10]:
```
F = np.array(
    [
        [2.70, 3.13, 3.55, 3.84],
        [2.74, 3.17, 3.59, 3.88],
        [2.79, 3.24, 3.70, 3.99],
        [3.08, 3.50, 3.94, 4.22],
        [4.16, 4.52, 5.09, 5.45],
        [4.54, 5.27, 6.02, 6.39],
        [11.29, 11.27, 10.52, 10.14],
    ]
)
print("Forward rates matrix\n", F)

N_RATINGS = P0.shape[0]
print("Number of possible Ratings =", N_RATINGS)
N_YEARS = F.shape[1]
print("Number of years in the forward rates matrix =", N_YEARS)
```

```
Forward rates matrix
 [[ 2.7   3.13  3.55  3.84]
 [ 2.74  3.17  3.59  3.88]
 [ 2.79  3.24  3.7   3.99]
 [ 3.08  3.5   3.94  4.22]
 [ 4.16  4.52  5.09  5.45]
 [ 4.54  5.27  6.02  6.39]
 [11.29 11.27 10.52 10.14]]
Number of possible Ratings = 8
Number of years in the forward rates matrix = 4
```

Once we have all this information, we can price the bond under usual methods. For the non-default ratings, this would be:

$$ V_i = C + \sum_{t=1}^{4} \frac{C}{(1 + \frac{F_{i,t}}{100})^t} + \frac{FACE}{(1 + \frac{F_{i,4}}{100})^4} $$

For the default state ("D" rating), this would be easier as it is simply the face value of the bond times the recovery rate:

$$ V_D = FACE \times Recovery \ rate $$

Let's put all this into code:

In [11]:
```
# Initialize bond values vector for each state (rating), including default
bond_values = np.zeros(N_RATINGS)
bond_values[N_RATINGS - 1] = D_RECOVERY/100 * FACE

# Calculate bond values for each non-default state
for r in range(0, N_RATINGS - 1):
    bond_values[r] = COUPON
    for t in range(0, N_YEARS):
        bond_values[r] = bond_values[r] + (COUPON + FACE * (N_YEARS - 1 == t)) /
            1 + (F[r, t] / 100)
        ) ** (t + 1)

# Thus, bond values for each rating are:
print("Bond values:\n ", bond_values)

# Expected value taking into account transition probabilities
pw_values = np.multiply(bond_values, P[RATINGS[CURR_RATING], :])
bond_val = np.sum(pw_values)

# Which yield a final bond value 1 year ahead of:
print("Bond value (one-year ahead): ", bond_val)
```

```
Bond values:
  [104.707 104.561 104.165 103.328  99.058  95.969  84.461  62.    ]
Bond value (one-year ahead):  98.7026456331422
```

3. Conclusion

Well done! We now know how to use Markov models and transition probabilities to value assets such as corporate bonds based on credit ratings. However, the usual problem we encounter in practice is how to determine these transition probabilities if they are not known ex ante. This is what we will explore in the next lesson, where we introduce **Hidden Markov Models (HMM)**.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

STOCHASTIC MODELING

MODULE 5 | LESSON 3

HIDDEN MARKOV MODELS (HMM)

Reading Time	80 minutes
Prior Knowledge	Linear Algebra, Maximum likelihood estimator, Monte Carlo

|**Keywords** |Markov Process, Markov chains, Transition Matrix

---

We have already seen the power of Markov chains in asset valuation taking into account changes in the 'state' of the world such as a change in rating. However, this process required knowledge of transition matrix, which is not an easy information to have. Here, we introduce **Hidden Markov Models (HMM)** as a tool that helps in this endeavour.

In [1]:
```
from IPython.display import VimeoVideo

# Bigger video
VimeoVideo("1121457333", h="3298dbabb7", width=700, height=450)
```

Out[1]:

1. Hidden Markov Models (HMM)

A Hidden Markov Model (HMM) is a Markov process that is split into two components: an observable component and an unobservable or "hidden" component that follows a Markov process. HMMs naturally describe setups where a stochastic system is observed through noisy measurements, for instance stock prices, that are affected by an unobserved economic factor. This type of modelization is of broad application in many fields, from speech recognition to thermodynamics. Let's here focus on one financial application of HMMs.

A basic HMM contains the following components:

- A set of N states $\mathcal{S}=\{s_1,...,s_N\}$
- A transition probability matrix P
- A sequence of T, possibly vector-valued, observations $\mathcal{Y}_T=\{y_1,...,y_T\}$
- A sequence of observation marginal likelihoods $f(y_t|s_t=i)$ for each $i=1,..,N$.
- An initial probability distribution $\pi=\{\pi_1,...,\pi_N\}$.

An important assumption is that the hidden Markov process is independent of past observations $\mathcal{Y}_{t-1}$, i.e., $\mathbb{P}\{s_t=j|s_t=i,\mathcal{Y}_{t-1}\}=\mathbb{P}\{s_t=j|s_t=i\}=p_{ij}$.

Conceptually, you could link this to the efficient market hypothesis, whereby current asset prices reflect all (past) available information.

This type of setup is also labeled in some instances as a Markov regime-switching model, where the observable output y_t features a marginal distribution whose parameters change with the realization of an unobservable state.

Let's see this with an example of Hidden Markov Process.

Suppose that $$ y_t=\mu_t+\varepsilon_t $$
where $\mu_t = \mu_j$ if $s_t=j$, $\mathcal{S}=\{1,...N\}$, S_t is Markovian, and ε_t is i.i.d. $N(0,\sigma^2)$.

We are interested in making inference about the probability of being in each state s_i at each date t, as well as estimating the model parameters of the transition matrix P and the vector $(\mu_1,...,\mu_N,\sigma)$. We collect all the parameters in a vector θ and assume for now that we know its value with certainty.

We can build the log-likelihood function of the process:

$$ \begin{align} \mathcal{L}(\theta) = \sum_{t=1}^T log\ f(y_t|\mathcal{Y}_{t-1};\theta) \end{align} $$

where $$ \begin{align} f(y_t|\mathcal{Y}_{t-1};\theta) = \sum_{i=1}^N \mathbb{P}(s_t=i|\mathcal{Y}_{t-1};\theta) \times \phi_{t}(i) \end{align} $$

with $\phi_{t}(i)=\phi\left(\frac{y_t-\mu_i}{\sigma}\right)$. $\phi(.)$ denotes the standard normal probability density function.

The main problem arises from the evaluation of $\mathbb{P}(s_t=i|\mathcal{Y}_{t-1};\theta)$, which would usually require an overwhelming amount of computations for each potential path $\mathcal{Y}_{t-1}$ and for each period t. This rules out estimating directly θ by Maximum Likelihood.

2. Hamilton filter

One potential approach to reach a solution is to denote $\xi_{t|t}(j)=\mathbb{P}(s_t=j|\mathcal{Y}_{t};\theta)$. Then, to estimate the model, we can rely on a recursive algorithm that sets the optimal forecasts $\xi_{t+1|t}(j)=\mathbb{P}(s_{t+1}=j|\mathcal{Y}_{t};\theta)$:

$$ \xi_{t|t}(i) = \frac{\xi_{t|t-1}(i)\phi_{t}(i)}{f(y_{t}|\mathcal{Y}_{t-1};\theta)} $$
$$ \xi_{t+1|t}(j) = \sum_{i=1}^N p_{ij} \xi_{t|t}(i) $$

This procedure is called the **Hamilton filter**, developed by Hamilton (1990). To initialize the recursion, we can set $\xi_{1|0}(i)=1/N$ or, if we have some guess for the initial distribution, $\xi_{1|0}(i)=\pi_i$. Another alternative is to include it in the vector of parameters to estimate, under the usual constraints for a probability distribution.

Then, to estimate the model, we would set initial parameters θ^0 that would allow us to recover each $\xi_{t+1|t}(j)$ and evaluate the log-likelihood function, then iterate with new guess θ^1 and so on until convergence.

Notice that we can make forecasts on the observable process y_{t+1} by exploiting the expressions for $\xi_{t+1|t}(j)$ once we have estimated the model parameters.

3. Hamilton filter in practice

Let's bring to practice this estimation by simulating a time series governed by a Hidden Markov process. We first build a simulated time series and then estimate the model by maximum likelihood, applying the Hamilton filter within the evaluation step.

The idea here is to see how the Hamilton filter allows us to recover otherwise unknown probabilities of different states. To start with, let's go over a tractable example.

3.1. A tractable example

Let's first see how the **Hamilton filter** works by doing an example we can replicate 'by hand'. To start with, we need to define the ingredients of the Markov model.

Ingredients of a simple HMM

Suppose we have a time series with just 3 observations ($T=3$), such that:

$$ Y = (y_1, y_2, y_3) = (-0.85, 0.4, -0.2) $$

We assume that this time series is generated by a Hidden Markov process such that:
$$ y_t = \epsilon_t $$

where the distribution of ϵ_t is normal with mean μ_i and variance σ^2_i. There are 2 states of the world (e.g., recession and expansion) such that:
$$ \boldsymbol{\mu} = (-1, 1) \ \ \text{ and } \ \ \boldsymbol{\sigma} = (0.8, 0.8) $$

Moreover, the matrix of transition probabilities, P, is given by:

$$ \mathbf{P} = \begin{pmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{pmatrix} $$

Lastly, the initial probability distribution for each state, π, is:

$$ \boldsymbol{\pi} = (0.2, 0.8) $$

Remember that, for each observation y_t, the likelihood of observing it in state s is given by:

$$ f(y_t | S_t = s) = \frac{1}{\sqrt{2\pi\sigma_s^2}} \exp\left(-\frac{(y_t - \mu_s)^2}{2\sigma_s^2}\right) $$

And, thus, the total log-likelihood is given by::

$$ \mathcal{L}(\boldsymbol{\theta}; Y) = \log f(y_3|y_2, y_1; \boldsymbol{\theta}) + \log f(y_2|y_1; \boldsymbol{\theta}) + \log f(y_1; \boldsymbol{\theta}) $$

where:

$$ f(y_1; \boldsymbol{\theta}) = P(s_1 = 1; \boldsymbol{\theta}) \times \frac{1}{\sigma_1} \phi\left(\frac{y_1 - \mu_1}{\sigma_1}\right) + P(s_1 = 2; \boldsymbol{\theta}) \times \frac{1}{\sigma_2} \phi\left(\frac{y_2 - \mu_2}{\sigma_2}\right) $$
$$ f(y_2|y_1; \boldsymbol{\theta}) = P(s_2 = 1|y_1; \boldsymbol{\theta}) \times \frac{1}{\sigma_1} \phi\left(\frac{y_2 - \mu_1}{\sigma_1}\right) + P(s_2 = 2|y_1; \boldsymbol{\theta}) \times \frac{1}{\sigma_2} \phi\left(\frac{y_2 - \mu_2}{\sigma_2}\right) $$
$$ f(y_3|y_2, y_1; \boldsymbol{\theta}) = P(s_3 = 1|y_2, y_1; \boldsymbol{\theta}) \times \frac{1}{\sigma_1} \phi\left(\frac{y_3 - \mu_1}{\sigma_1}\right) + P(s_3 = 2|y_2, y_1; \boldsymbol{\theta}) \times \frac{1}{\sigma_2} \phi\left(\frac{y_3 - \mu_2}{\sigma_2}\right) $$

We can simplify these expressions given that:

$$ \frac{1}{\sigma_1} \phi\left(\frac{y_1 - \mu_1}{\sigma_1}\right) = \frac{1}{0.8} \phi\left(\frac{(-0.85 - (-1))}{0.8}\right) \approx 0.49 $$

Analogously:

$$ \frac{1}{\sigma_2} \phi\left(\frac{y_1 - \mu_2}{\sigma_2}\right) = 0.034 $$

$$ \frac{1}{\sigma_1} \phi\left(\frac{y_2 - \mu_1}{\sigma_1}\right) = 0.108 $$

$$ \frac{1}{\sigma_2} \phi\left(\frac{y_2 - \mu_2}{\sigma_2}\right) = 0.376 $$

$$ \frac{1}{\sigma_1} \phi\left(\frac{y_3 - \mu_1}{\sigma_1}\right) = 0.302 $$

$$ \frac{1}{\sigma_2} \phi\left(\frac{y_3 - \mu_2}{\sigma_2}\right) = 0.162 $$

which simplies the different components of the log-likelihood to:

$$ f(y_1; \boldsymbol{\theta}) = P(s_1 = 1; \boldsymbol{\theta}) \times 0.49 + P(s_1 = 2; \boldsymbol{\theta}) \times 0.034 $$

$$ f(y_2|y_1; \boldsymbol{\theta}) = P(s_2 = 1|y_1; \boldsymbol{\theta}) \times 0.108 + P(s_2 = 2|y_1; \boldsymbol{\theta}) \times 0.376 $$

$$ f(y_3|y_2, y_1; \boldsymbol{\theta}) = P(s_3 = 1|y_2, y_1; \boldsymbol{\theta}) \times 0.302 + P(s_3 = 2|y_2, y_1; \boldsymbol{\theta}) \times 0.162 $$

Now its time to estimate the state probabilities, $\xi_{t+1|t}(s)$, using Hamilton's filter. Starting in $t=1$ this implies:

Step 1: Initialize with given parameters (t=1)

$$ P(s_1 = s; \boldsymbol{\theta}) = \pi_s = \xi_{1|0}(s) $$

Step 2: Compute marginal likelihood

$$ f(y_1; \boldsymbol{\theta}) = \pi_1 \times 0.49 + \pi_2 \times 0.034 = 0.2 \times 0.49 + 0.8 \times 0.034 = 0.126 $$

Step 3: Filtering (applying Bayes theorem)

$$ P(s_1 = s|y_1; \boldsymbol{\theta}) = \frac{P(s_1 = s; \boldsymbol{\theta}) \times P(y_1|s_1 = i; \boldsymbol{\theta})}{P(y_1; \boldsymbol{\theta})} = \frac{\pi_s \times \phi\left(\frac{y_1-\mu_s}{\sigma_s}\right)/\sigma_i}{f(y_1; \boldsymbol{\theta})} = \xi_{1|1}^{(s)} $$

As a side note, remember Bayes Theorem:

$$ P(A|B) = \frac{P(B|A)P(A)}{P(B)} $$

Step 4: Predict next period

$$ P(s_2 = s|y_1; \boldsymbol{\theta}) = p_{1s} \xi_{1|1}(1) + p_{2s} \xi_{1|1}(2) = \xi_{2|1}(s) $$

Following this process, we get:

$$ \xi_{1|1}(1) = 0.781; \quad \xi_{1|1}(2) = 0.219; \quad \xi_{2|1}(1) = 0.668; \quad \xi_{2|1}(2) = 0.332 $$

Once we have this for $t=1$, we can analogously do it for $t=2$, repeating the same process:

$$ f(y_2|y_1; \boldsymbol{\theta}) = \xi_{2|1}(s=1) \times 0.108 + \xi_{2|1}(s=2) \times 0.376 = 0.197 $$

$$ P(s_2 = s|y_2, y_1; \boldsymbol{\theta}) = \frac{P(s_2 = s|y_1; \boldsymbol{\theta}) \times P(y_2|s_2 = s, y_1; \boldsymbol{\theta})}{P(y_2|y_1; \boldsymbol{\theta})} = \frac{\xi_{2|1}(s) \times \phi\left(\frac{y_2-\mu_s}{\sigma_s}\right)/\sigma_s}{f(y_2|y_1; \boldsymbol{\theta})} = \xi_{2|2}(s) $$

$$ P(s_3 = s|y_2, y_1; \boldsymbol{\theta}) = p_{1s} \xi_{2|2}(s=1) + p_{2s} \xi_{2|2}(s=2) = \xi_{3|2}(s) $$

which yields:

$$ \xi_{2|2}(1) = 0.366; \quad \xi_{2|2}(2) = 0.634; \quad \xi_{3|2}(1) = 0.42; \quad \xi_{3|2}(2) = 0.58 $$

Finally, for period $t=3$ we repeat again:

$$ f(y_3|y_2, y_1; \boldsymbol{\theta}) = \xi_{3|2}(1) \times 0.302 + \xi_{3|2}(2) \times 0.162 = 0.221 $$

$$ P(s_3 = i|y_3, y_2, y_1; \boldsymbol{\theta}) = p_{1s} \xi_{3|2}(1) + p_{2s} \xi_{3|2}(2) = \xi_{3|3}(s) $$

$$ \xi_{3|3}(1) = 0.575; \quad \xi_{3|3}(2) = 0.425 $$

This would produce a log-likelihood given by:

$$ \mathcal{L}(\boldsymbol{\theta}; Y) = \log f(y_1; \boldsymbol{\theta}) + \log f(y_2|y_1; \boldsymbol{\theta}) + \log f(y_3|y_2, y_1; \boldsymbol{\theta}) = \log(0.126) + \log(0.197) + \log(0.221) = -5.202 $$

So far, during this tedious process, we have implemented the Hamilton procedure just to evaluate the 'forward pass' of the model. That is, given some initial parameters, we can use Hamilton's procedure to estimate the probabilities of being at each state and evaluate the overall log-likelihood.

The next natural step would be to recursively estimate model parameters seeking maximum likelihood. Let's implement this in coding.

3.2. Hamilton filter example in python

Next, we are going to verify that using a number of simple functions defined in python we can yield the same results as in the previous hand-written example.

For starters, let's import some useful libraries:

In [2]:
```
import numpy as np
from scipy.optimize import minimize
from scipy.stats import norm
import matplotlib.pyplot as plt
```

And let's define some functions:

In [3]:
```
def emission_prob(y, mu, sigma):
    """Calculate emission probability: f(y_t | s_t = i)"""
    return norm.pdf(y, loc=mu, scale=sigma)


def hamilton_filter(y, mu, sigma, P, pi):
    """
    Hamilton Filter implementation

    Parameters:
    y: observations (T,)
    mu: values for each state (n_states,)
    sigma: std devs for each state (n_states,)
    P: transition matrix (n_states, n_states)
    pi: initial probabilities (n_states,)

    Returns:
    filtered_probs: filtered probabilities (T, n_states)
    predicted_probs: predicted probabilities (T, n_states)
    log_likelihood: total log likelihood
    marginal_likelihoods: marginal likelihoods at each step
    """
    T = len(y)
    n_states = len(mu)

    # Storage
    filtered_probs = np.zeros((T, n_states))
    predicted_probs = np.zeros((T, n_states))
    marginal_likelihoods = np.zeros(T)

    # Initialize with prior
    predicted_probs[0] = pi

    for t in range(T):
        # Step 1: Calculate emission probabilities for all states (f( ))
        emissions = np.array([emission_prob(y[t], mu[i], sigma[i])
                              for i in range(n_states)])

        # Step 2: Calculate marginal likelihood
        marginal_likelihoods[t] = np.sum(predicted_probs[t] * emissions)

        # Step 3: Update (filter) - Bayes rule
        filtered_probs[t] = (predicted_probs[t] * emissions) / marginal_likelihoo

        # Step 4: Predict next period (if not last observation)
        if t < T - 1:
```

```
            predicted_probs[t + 1] = P.T @ filtered_probs[t]

    # Calculate log likelihood
    log_likelihood = np.sum(np.log(marginal_likelihoods))

    return filtered_probs, predicted_probs, log_likelihood, marginal_likelihoods
```

With these we should be able to replicate the hand-written example, given the initial parameters:

In [4]:
```
# Initial data
y = np.array([-0.85, 0.4, -0.2])
mu = np.array([-1.0, 1.0])
sigma = np.array([0.8, 0.8])
P = np.array([[0.8, 0.2],
              [0.2, 0.8]])
pi = np.array([0.2, 0.8])
```

In [5]:
```
# Run Hamilton filter
filtered_probs, predicted_probs, log_likelihood, marginals = hamilton_filter(y,
```

Let's check the results and whether they align with previous ones:

In [6]:
```
print(f"Total log-likelihood: {log_likelihood:.3f}")
print("Predicted probabilities (before observing y_t):")
for t in range(len(y)):
    if t == 0:
        print(f"  ξ_{t+1}|{t} = {predicted_probs[t]} (initial π)")
    else:
        print(f"  ξ_{t+1}|{t} = [{predicted_probs[t,0]:.3f}, {predicted_probs[t,
```

```
Total log-likelihood: -5.210
Predicted probabilities (before observing y_t):
  ξ_1|0 = [0.2 0.8] (initial π)
  ξ_2|1 = [0.668, 0.332]
  ξ_3|2 = [0.420, 0.580]
```

As you can see, we obtain essentially the same numbers (except for some rounding error) in a much more efficient way. This allows us to recursively estimate model parameters (i.e., θ vector) by maximizing log-likelihood (or, equivalently, minimizing negative-log-likelihood).

3.3. Estimating parameters by maximizing log-likelihood

Now we can setup functions to obtain the log-likelihood and perform estimation of model parameters. The iterative method to estimate these parameters is usually known as **Expectation-Maximization (EM) algorithm**:

In [7]:
```
def negative_log_likelihood(params, y):
    """
    Negative log likelihood function for optimization
    We MINIMIZE this function to MAXIMIZE the likelihood

    Parameters format: [mu1, mu2, sigma1, sigma2, p11, p22, pi1]
    """
```

```
    mu1, mu2, sigma1, sigma2, p11, p22, pi1 = params

    # Basic constraints to avoid crazy results
    if sigma1 <= 0 or sigma2 <= 0:
        return 1e10
    if sigma1 >= 1 or sigma2 >= 1:
        return 1e10
    if not (0 <= p11 <= 1) or not (0 <= p22 <= 1):
        return 1e10
    if not (0 <= pi1 <= 1):
        return 1e10

    # Build parameter arrays
    mu = np.array([mu1, mu2])
    sigma = np.array([sigma1, sigma2])
    P = np.array([[p11, 1-p11],
                  [1-p22, p22]])
    pi = np.array([pi1, 1-pi1])

    # Run Hamilton filter to get log-likelihood
    try:
        _, _, log_likelihood, _ = hamilton_filter(y, mu, sigma, P, pi)
        return -log_likelihood
    except:
        return 1e10
```

In [8]:

```
def estimate_mle(y, initial_guess=None):
    """
    Maximum Likelihood Estimation using Hamilton Filter

    We find: θ* = argmax ℒ(θ) = argmin(-ℒ(θ))
    Note that maximizing log-likelihood is equivalent to minimizing negative-log
    """
    if initial_guess is None:
        initial_guess = [-1.0, 1.0, 0.8, 0.8, 0.8, 0.8, 0.2]

    print(f"Initial guess: {initial_guess}")


    # Try optimization
    result = minimize(negative_log_likelihood, initial_guess, args=(y,), method=

    print(f"Optimization result: {result.success}")
    if not result.success:
        print(f"Warning: {result.message}")
        print("Results may be unreliable with only 3 observations!")

    # Extract estimated parameters
    mu1, mu2, sigma1, sigma2, p11, p22, pi1 = result.x

    mu_est = np.array([mu1, mu2])
    sigma_est = np.array([sigma1, sigma2])
    P_est = np.array([[p11, 1-p11], [1-p22, p22]])
    pi_est = np.array([pi1, 1-pi1])

    # Maximum log-likelihood is negative of minimized value
    max_log_likelihood = -result.fun

    return mu_est, sigma_est, P_est, pi_est, max_log_likelihood
```

So now we can estimate parameters from the data, given initial guesses:

In [9]:
```
mu_est, sigma_est, P_est, pi_est, max_log_likelihood = estimate_mle(y)
```

```
Initial guess: [-1.0, 1.0, 0.8, 0.8, 0.8, 0.8, 0.2]
Optimization result: False
Warning: Desired error not necessarily achieved due to precision loss.
Results may be unreliable with only 3 observations!
```

In [10]:
```
print(f"\n- ESTIMATED PARAMETERS -")
print("-" * 25)
print(f"μ₁  {mu_est[0]:.3f}")
print(f"μ₂  {mu_est[1]:.3f}")
print(f"σ₁  {sigma_est[0]:.3f}")
print(f"σ₂  {sigma_est[1]:.3f}")
print(f"π₁  {pi_est[0]:.3f}")
print(f"π₂  {pi_est[1]:.3f}")
```

```
- ESTIMATED PARAMETERS -
-------------------------
μ₁  -0.318
μ₂  0.044
σ₁  0.409
σ₂  0.274
π₁  0.971
π₂  0.029
```

As we have highlighted in the previous code, it's likely that our estimation does not yield the desired results. This is somewhat obvious at first sight, since the amount of data we can work with is really reduced (just 3 observations!). But for now, you know how to implement (and trace back) a Hamilton filter in a Hidden Markov Model.

4. Conclusion

Well done! Implementing HMM in practice clearly presents challenges. We have seen this already with the practical implementation of Hamilton's filter. However, the idea is that you understand the basics behind the implementations. Once we put things into coding, the process is more friendly.

Hamilton filter is the first step towards being able to forecast probabilities of regime switching, which has important implications for stock market forecasting. This is what we are going to see in the next lesson.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

STOCHASTIC MODELING

MODULE 5 | LESSON 4

EM ALGORITHM AND REGIME SWITCHING MODELS

Reading Time	60 minutes
Prior Knowledge	Linear Algebra, Maximum likelihood estimator, Monte Carlo

|**Keywords** | HMM, Markov chains, Transition Matrix

Let's put our newly-acquired knowledge of HMM to use. In this lesson we will develop a regime-switching model for the VIX index, a popular measure of overall stock market's expectation of volatility. The VIX index is developed by CBOE and constructed using implied volatility from S&P 500 index options. Thus, we can interpret it as a forward looking measure of market regime (i.e., calm vs. stressed periods), which can have wide applications for trading.

In [1]:
```
!pip install pandas_datareader
```

```
Requirement already satisfied: pandas_datareader in /usr/local/lib/python3.11/sit
e-packages (0.10.0)
Requirement already satisfied: lxml in /usr/local/lib/python3.11/site-packages (f
rom pandas_datareader) (5.3.1)
Requirement already satisfied: pandas>=0.23 in /usr/local/lib/python3.11/site-pac
kages (from pandas_datareader) (2.2.3)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.11/site
-packages (from pandas_datareader) (2.32.3)
Requirement already satisfied: numpy>=1.23.2 in /usr/local/lib/python3.11/site-pa
ckages (from pandas>=0.23->pandas_datareader) (2.1.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.1
1/site-packages (from pandas>=0.23->pandas_datareader) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/site-pac
kages (from pandas>=0.23->pandas_datareader) (2025.1)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/site-p
ackages (from pandas>=0.23->pandas_datareader) (2025.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python
3.11/site-packages (from requests>=2.19.0->pandas_datareader) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/site-pac
kages (from requests>=2.19.0->pandas_datareader) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/si
te-packages (from requests>=2.19.0->pandas_datareader) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/si
te-packages (from requests>=2.19.0->pandas_datareader) (2025.1.31)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/site-package
s (from python-dateutil>=2.8.2->pandas>=0.23->pandas_datareader) (1.17.0)
WARNING: Running pip as the 'root' user can result in broken permissions and conf
licting behaviour with the system package manager, possibly rendering your system
unusable. It is recommended to use a virtual environment instead: https://pip.pyp
a.io/warnings/venv. Use the --root-user-action option if you know what you are do
ing and want to suppress this warning.

[notice] A new release of pip is available: 25.0.1 -> 26.1.1
[notice] To update, run: pip install --upgrade pip
```

In [2]:
```python
# Libraries needed
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import pandas_datareader.data as web
from scipy.stats import norm
from scipy.optimize import minimize
```

1. Gathering and handling VIX Data

Before jumping to the methodological implementation, let's gather our core data for VIX index prices during the last 10 years. We are going to gather this data daily from the FRED (Federal Reserve Economic Data):

In [3]:
```python
vix_data = web.get_data_fred('VIXCLS', start='2015-01-01', end='2024-12-31')
```

This gets us the data on VIX levels, but for HMM we would likely rather **work with changes rather than levels**. This has a number of advantages:

- Changes are closer to stationary

- Mean-reverting around zero, which allows for better time comparison of regimes.
- Changes in VIX are more representative of actual trading gains if we want to eventually develop a trading strategy for VIX

Unfortunately, this can also mean that regime thresholds are less obvious and difficult to interpret.

In [4]:
```
vix_changes = vix_data.diff().dropna()
```

In [ ]:
```
# Let's store the time series in a variable y for easier handling over the noteb
y = vix_changes.iloc[:,0].values

# Basic visualization
plt.figure(figsize=(12, 6))

plt.subplot(2, 1, 1)
plt.plot(vix_data.index, vix_data.iloc[:,0])
plt.title('VIX Levels')
plt.ylabel('VIX')

plt.subplot(2, 1, 2)
plt.plot(vix_changes.index, vix_changes.iloc[:,0])
plt.title('VIX Changes')
plt.ylabel('Change')

plt.tight_layout()
plt.show()
```

2. Setting up the regime-switching HMM

For the next step, we need to set up the HMM. This will require us to consider some initial parameter guesses (i.e., remember μ_1, μ_2, σ_1, σ_2, ...).

For illustrative purposes, we will set these initial parameter values based on the history of observed VIX changes. Notice, however, **that this is introducing look-ahead bias into the whole process**. This is because we are using information about the whole history of VIX that would not be known at each point in time. For example, say we consider a change of +/-10 in the VIX (such the one experienced in 2009) as the threshold for defining the regime as "stressed" period. This is information that we would never be able to know prior to that date and, thus, not 'fair game' for our trading strategy. Indeed, the perfectly correct solution would be to use a rolling window over the data to initialize these parameters based on the historical past distribution at each moment in time. For practicality, we are going to obviate this for now and focus on the rest of the implementation:

In [ ]:
```
# Defined initial parameters based on data (see caveats)

# Regime 1 (Calm): small changes around zero
mu1 = 0.0  # Assume calm regime centered at zero

# Regime 2 (Stressed): large positive changes (high uncertainty)
```

```
stressed_threshold = np.percentile(y, 85)  # Top 15% of changes
mu2 = np.mean(y[y > stressed_threshold])

# Volatilities based on regime classification
calm_changes = y[np.abs(y) <= stressed_threshold]
stressed_changes = y[y > stressed_threshold]
sigma1 = np.std(calm_changes)
sigma2 = np.std(stressed_changes)

# Transition probabilities (default ad hoc assumptions)
p11 = 0.95  # Calm periods very persistent
p22 = 0.75  # Stressed periods less persistent

# Initial state probability
pi1 = 0.8   # Usually start in calm regime

# Combine into initial guess vector
initial_guess = [mu1, mu2, sigma1, sigma2, p11, p22, pi1]
```

3. Hamilton filter

Now, under this setup, we can implement the Hamilton filter procedure we saw in previous lessons to identify the probabilities of each period leading to "calm" versus "stressed" period next. Let's do it!

Note that in essence the following code is doing exactly the same of what we did in the previous example. However, we have modified some things slightly, including bounds to the parameters to preserve convergence

In [ ]:
```
EPS = 1e-12  # tiny floor to avoid underflow / log(0)


def emission_prob(y, mu, sigma):
    """Calculate emission probability: f(y_t | s_t = i)"""
    return norm.pdf(y, loc=mu, scale=sigma)


def hamilton_filter(y, mu, sigma, P, pi):
    T = len(y)
    n_states = len(mu)

    filtered_probs = np.zeros((T, n_states))
    predicted_probs = np.zeros((T, n_states))
    marginal_likelihoods = np.zeros(T)

    # Normalize just in case the initial pi isn't exactly on the simplex
    pi = np.asarray(pi, float)
    pi = pi / pi.sum()
    predicted_probs[0] = pi

    for t in range(T):
        # 1) Emissions (floored)
        emissions = np.array([emission_prob(y[t], mu[i], sigma[i]) for i in rang
        emissions = np.maximum(emissions, EPS)

        # 2) Marginal likelihood (floored)
```

```
        ml = float(np.dot(predicted_probs[t], emissions))
        ml = max(ml, EPS)
        marginal_likelihoods[t] = ml

        # 3) Filter (Bayes) + renormalize defensively
        post = (predicted_probs[t] * emissions) / ml
        s = post.sum()
        filtered_probs[t] = post / s if s > 0 else np.full(n_states, 1.0/n_state

        # 4) Predict next period
        if t < T - 1:
            pred = P.T @ filtered_probs[t]  # keep your convention
            pred = np.clip(pred, EPS, 1.0)
            predicted_probs[t + 1] = pred / pred.sum()

    log_likelihood = np.sum(np.log(marginal_likelihoods))
    return filtered_probs, predicted_probs, log_likelihood, marginal_likelihoods


def negative_log_likelihood(params, y):
    mu1, mu2, sigma1, sigma2, p11, p22, pi1 = params

    # Basic constraints
    if sigma1 <= 0 or sigma2 <= 0:
        return 1e10
    if not (0 <= p11 <= 1) or not (0 <= p22 <= 1):
        return 1e10
    if not (0 <= pi1 <= 1):
        return 1e10

    mu = np.array([mu1, mu2], float)
    sigma = np.array([sigma1, sigma2], float)
    P = np.array([[p11, 1 - p11],
                  [1 - p22, p22]], float)
    pi = np.array([pi1, 1 - pi1], float)

    try:
        _, _, log_likelihood, _ = hamilton_filter(y, mu, sigma, P, pi)
        if not np.isfinite(log_likelihood):
            return 1e10
        return -log_likelihood
    except Exception:
        return 1e10



def estimate_mle(y, initial_guess):
    bounds = [
        (None, None),       # mu1
        (None, None),       # mu2
        (1e-6, None),       # sigma1 > 0
        (1e-6, None),       # sigma2 > 0
        (1e-6, 1 - 1e-6),   # p11 in (0,1)
        (1e-6, 1 - 1e-6),   # p22 in (0,1)
        (1e-6, 1 - 1e-6),   # pi1 in (0,1)
    ]

    result = minimize(negative_log_likelihood, initial_guess,
                      args=(y,), method='L-BFGS-B', bounds=bounds)
```

```
    mu1, mu2, sigma1, sigma2, p11, p22, pi1 = result.x
    mu_est = np.array([mu1, mu2])
    sigma_est = np.array([sigma1, sigma2])
    P_est = np.array([[p11, 1 - p11], [1 - p22, p22]])
    pi_est = np.array([pi1, 1 - pi1])
    max_log_likelihood = -result.fun

    return mu_est, sigma_est, P_est, pi_est, max_log_likelihood
```

4. Estimation results

Let's run our **Expectation-Maximization (EM) algorithm** to estimate these parameters (careful, this could take some time!):

In [ ]:
```
initial_guess = [
    mu1,   # mu1
    mu2,   # mu2
    sigma1,   # sigma1  (low-vol regime)
    sigma2,   # sigma2  (high-vol regime)
    p11,   # p11     (persist in low-vol)
    p22,   # p22     (persist in high-vol)
    pi1    # pi1     (initial prob of state 1)
]

# 2) Estimate parameters via MLE
mu_est, sigma_est, P_est, pi_est, max_ll = estimate_mle(y, initial_guess)

print("Estimated parameters")
print("mu:", mu_est)
print("sigma:", sigma_est)
print("P:\n", P_est)
print("pi:", pi_est)
print("Max log-likelihood:", max_ll)
```

Now with these estimated parameters we can re-run the Hamilton filter to obtain, at each date t, the predicted probabilities of a regime ocurring next date ($t+1$). Specifically, using the Hamilton filter we will next compute:

1. **Filtered probabilities** ($Pr(s_t = i | y_{1:t})$): These are *posterior probabilities*, in the sense that they tell us how likely is that we are in state s today given all data up to today.

2. **Predicted probabilities** ($Pr(s_t = i | y_{1:t-1})$): These are *prior probabilities*, as they use the transition matrix to inform us about the change of being in state s today given what we know until yesterday.

In [ ]:
```
filtered_probs, predicted_probs, loglike, _ = hamilton_filter(y, mu_est, sigma_e
```

We can use predicted probabilities to design trading strategies. For example, we will go long VIX if the predicted probabity of being at a high volatility state tomorrow (i.e., rises in VIX index) is very high. But we can also impose other rules like, conditional on being on a stressed state today (i.e, high VIX level), we can short VIX if the predicted probability of being in a calm state tomorrow is very high.

5. Trading strategy and backtest

Let's leverage the use of predicted probabilities to design a trading strategy on the VIX. For now we will keep things simple and base our strategy on 2 simple rules:

1. Conditional on high chance of being in a calm state, **go long VIX** if the predicted probability of being in a stressed state tomorrow is higher than some threshold.

2. Conditional on high chance of being in a stressed state, **go short VIX** if the predicted probability of being in a calm state tomorrow is higher than some threshold.

In order to design such an strategy, we need to compute $Pr(s_{t+1} | y_{1:t})$, that is, the forecast for tomorrow's state of the world given information up to today. This is slightly different from the predicted probability we already have, but nothing we cannot compute. Mathematically, this simply entails propagating the distribution through transition matrix P:

$$ Pr(s_{t+1} | y_{1:t}) = P^T Pr(s_{t} = i | y_{1:t}) $$

In [ ]:
```
next_probs = (P_est.T @ filtered_probs.T).T
```

Now let's create a convenient dataframe to then calculate the signals of our trading strategy:

In [ ]:
```
# Use the same index dates as predictions
df = pd.DataFrame(index=vix_changes.index)

# Add VIX data in levels (rather than changes) and calculate returns for later o
df['VIX'] = vix_data.iloc[:,0]
df['VIX_ret'] = df['VIX'].pct_change()

# Include data on filtered probabilities
df['filt_s0'] = filtered_probs[:,0]
df['filt_s1'] = filtered_probs[:,1]

# Include data on predicted probabilities
df['prob_s0'] = next_probs[:,0]
df['prob_s1'] = next_probs[:,1]

# Shift by 1 the data on probs, in order to align todays's prediction of tomorro
df['prob_s0'] = df['prob_s0'].shift(1)
df['prob_s1'] = df['prob_s1'].shift(1)

# Drop resulting missing values
df = df.dropna()
```

And our trading signal and positions vector. The trading rules are described above, but we may also need a vector indicating the position that we are holding at every date. In

this case, we will hold a position until there is a change in the signal and assume we exit the position at the close price of that date. This is arguably very simplistic, but for now it will work well to illustrate things.

In [ ]:
```
df["signal"] = 0

# Conditional on high probability of being at a calm regime, go long if high pre
df.loc[(df['filt_s0'] > 0.5) & (df["prob_s1"] > 0.75), "signal"] = 1

# Conditional on high probability of being at a stressed regime, go short if hig
df.loc[(df['filt_s1'] > 0.5) & (df["prob_s0"] > 0.75), "signal"] = -1


df["position"] = (
    df["signal"]
      .replace(0, np.nan)   # treat 0 as "no new trade"
      .ffill()              # carry last non-zero forward
      .fillna(0)            # still 0 before first signal appears
      .astype(int))
```

If we further assume that the returns to holding cash are zero, we can simply calculate strategy returns as follows:

In [ ]:
```
df['strat_rets'] = df['position']* df['VIX_ret']
```

It is also true that the VIX index that we are considering here is not exactly tradable, but there are many ETFs closely tracking it with minimal trading costs. Let's visualize how our very simple strategy performs versus a buy and hold on the VIX:

In [ ]:
```
# Calculate cumulative returns
df["cum_vix"] = (1 + df["VIX_ret"]).cumprod()
df["cum_strat"] = (1 + df["strat_rets"]).cumprod()
```

In [ ]:
```
# Plot them!
plt.figure(figsize=(12,6))
plt.plot(df.index, df["cum_vix"], label="VIX Buy & Hold", color="black", alpha=0
plt.plot(df.index, df["cum_strat"], label="HMM Strategy", color="blue", linewidt
plt.title("Cumulative Returns: VIX vs HMM Strategy")
plt.xlabel("Date")
plt.legend()
plt.grid(True)
plt.show()
```

Unfortunately, is seems that our strategy does not perform too well. You can go back and play around with the different thresholds for the signals, but it seems that this very simple approach is not enough. But this was kind of expected, wasn't it?

4. Conclusion

Well done! In this module we have implemented in practice a very simple regime change model for the VIX index and developed a trading strategy based on the model. Of course, the results where not very promising, as one would expect.

In the next module we will continue to explore applications of HMM for finance, hopefully with better results!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

MScFE 622 STOCHASTIC MODELING

Group Work Project # 1

See grading rubric here.

Scenario

Financial engineers can use volatility models to price options. As a trio (or duo) of quants, you will be tasked with pricing options using different models. The outputs of your work are used all day, every day by the options desk – a profitable part of the firm's market making. Do your job correctly, and you will be considered the 'rocket scientists' of the group. Do your job incorrectly, and well… let's not go there.

A long-time client of the bank is looking to purchase an OTC Asian option on the company SM. Remember that an Asian call option payoff at maturity is simply defined as:

$$\max(Avg\left(S_t\right) - K; 0) \quad for\ t = \{0, 1, ...T\}$$

(Remember to include the current price today, S_0 , in the calculation of the average St).

The client is not so sure about the maturity she wants to consider for the option. Fortunately, you have information on vanilla options traded in the market, with the underlying asset being the stock of SM (for SM Energy Company), in the attached Excel file. SM stock is currently trading at $232.90 USD:

Click here to download Option Data to complete the Group Work Project.

Tasks

Step 1

a. Initially, the client seems to be looking for a very short maturity for her derivative (around 15 days). This is a good job for Team Member A!

   Team member A needs to calibrate a classic Heston (1993) model (without jumps) to the observed market prices for both call and put options. Use the Lewis (2001) approach with a regular MSE error function. For the moment, consider a

constant annual risk-free rate of 1.50%. Assume 1 year has 250 trading days. For the case of put options, note that you can either:

i. Use the Lewis (2001) closed form for put options that you have on the paper: Lewis, Alan L. "A simple option formula for general jump-diffusion and other exponential Lévy processes." (2001) https://papers.ssrn.com/sol3/papers.cfm?abstract_id=282110

ii. (preferred) Use put-call parity with the closed-form solution for the call option.

As part of the pricing team, Team member A needs to report the parameter values resulting from the calibration, describe the whole process for the other team members to know (to which maturity (-ies) you calibrated the model, error function considered,…), and briefly discuss the final calibration including graphs that properly illustrate the fit of the calibration.

b. In order to double-check the different model parameters resulting from a calibration, Team member B will repeat the same process as Team member A, but using the Carr-Madan (1999) pricing approach to calibrate the Heston (1993) model. Make sure that you repeat all the tasks in (a), including a discussion on why (or why not) you obtain similar values for the different parameters as via Lewis (2001). You can use put-call parity as well to calibrate to put option prices.

c. Using the calibrated parameters you consider appropriate from the previous tasks, Team member C will price the Asian call option for the client. In this case, the client wants an ATM Asian option with 20 days maturity.

As team member in charge of pricing for the client, make sure that you:

i. Obtain the 'fair price' of the instrument using Monte-Carlo methods in a risk-neutral setting. Make sure you perform enough simulations in Monte-Carlo. (*Hint: You may want to check some of the Derivative Pricing material for this*)

ii. As part of the bank's profit, you charge a 4% fee on the price to obtain the final price that the client will end up paying. Make sure to clearly state this in your report.

iii. Include in your report a brief but complete non-technical description (that a client can understand) on the process you undertook for pricing, including calibration steps and choices that you consider relevant.

Note: For Groups of 2, please use the responsibilities for Members A and C only.

© 2022 - WorldQuant University – All rights reserved.

Step 2

Unfortunately, the client seems hesitant about the short maturity considered in step 1. After giving it some thought, she thinks that an instrument with 60 days maturity would better adapt to her needs.

a. Team member C will now repeat Task (a) in Step 1 for the new case at hand (60-day maturity instrument) using a Heston model with jumps (i.e., Bates, 1996 model). Make sure you follow all the proper steps in the calibration of Bates, and that you explain them. Except for the mentioned change of the target maturity, use all other instructions from Task (a) in Step 1.
b. Team member A will repeat the previous Task (a) in Step 2 using Carr-Madan (1999) approach to Bates (1996) model. Except for the mentioned change of the target maturity, use all other instructions from Task (b) in Step 1.
c. Team member B will perform a pricing process similar to Task (c) of Step 1. In this case, rather than an OTC instrument, the client has decided she wants to buy a Put option on firm SM with 70 days maturity and moneyness of 95% (i.e., strike is 95% of the current price).

Note: For Groups of 2, please use the responsibilities for Members A and C only.

Step 3

Since the initial idea of our client was to purchase a very short-term maturity OTC instrument, we were not so concerned about the potential risks arising from future evolutions of interest rates. However, considering how volatile are the demands of the client (first asks for a very short-term Asian, then a mid-term regular put option… what will be next?), your boss asks the team to provide some insights on future interest rates. Specifically, **as a team**, you will need to:

a. Calibrate a CIR (1985) model considering current rates, describing the overall process.

   Since we as a bank operate mostly on a European setting, we will consider Euribor rates. Current rates and maturities are given in the following table:

Euribor 1 week	0.648 %
Euribor 1 month	0.679 %
Euribor 3 months	1.173 %
Euribor 6 months	1.809 %
Euribor 12 months	2.556 %

© 2022 - WorldQuant University – All rights reserved.

Make sure you properly build the term structure of the Euribor. Then, use the cubic spline method to interpolate weekly rates for a period of 12 months (1 years). Calibrate the model to interpolated term structure. Make sure you briefly describe the process, clearly show the output of the calibration, and discuss the different parameters obtained as well as the fit of your model under those parameters to market rates (include graphs).

b. Given the different CIR model parameters obtained in the previous step, simulate Euribor 12-month rates daily for a period of 1 year. Perform 100,000 Monte-Carlo simulations. Discuss the results obtained (include graphs) regarding:
   i. Select a level of confidence you are comfortable with, which is the range (max and min) that the 12-month Euribor can take in the next year?
   ii. What is the expected value of the 12-month Euribor in 1 year?
   iii. How will this expected number affect the pricing of your products in 1 year versus the current 12-month Euribor rate?

Step 4

As a team, the group will organize both the Python file for all the steps as well as work together on a report that contains the answers to all the questions above (except for the code) in a clear and organized manner (think about how you would present this to your boss).

Submission Requirements and Format

One team member submits the following on behalf of the entire group:

- **1 PDF document** containing ONLY the answers to the questions, EXCLUDING code
  - Use the available Report Template and fill out the required information in the first page.
- **1 zipped file** that contains:
  - Jupyter notebook that is executable
  - html of both the Jupyter notebook containing all code, results, and graphs

***Use Google Colab or GitHub to collaborate** in completing the executable Python program.*

NOTE: *The PDF must be uploaded **separately** from the zipped folder that includes any other types of files. This allows Turnitin to generate a similarity report.*

© 2022 - WorldQuant University – All rights reserved.

Rubric

Your instructor will evaluate your group submission for GWP1 using the following rubric:

Quantitative Analysis (Open-Ended Questions) 40 Points	Technical and Non-Technical Reports 30 Points	Writing and Formatting 20 Points
The group is able to apply results, formulas, and their knowledge of theory to real-life finance scenarios by doing the following: ● Providing all the necessary information to support their arguments. ● Presenting arguments that reflect group discussion and research. ● Using authoritative references to support a position and provide updated information. ● Concluding with practical takeaways for more insightful financial decision-making.	Technical reports contain 3 parts: 1) **code** for each question (be sure to explicitly state the question number), 2) the corresponding output of that **code**, and 3) **interpretations** and/or **recommended courses of action** that reasonably follow from those results. **Note**: Technical reports will include the technicalities of models, such as names, methods of estimation, parameter values, etc., and exclude generalities about the work done. It should NOT include names of Python code that were used.	A submission that looks professional should: ● Include the axes, labels, and scales in graphs. ● Be free of significant grammatical errors or typos. ● Be an organized, well-structured, and easy-to-read document. ● Include proper citations and a bibliography in MLA format.
	Non-technical reports contain 3 parts: 1) clear explanation of results; 2) the recommended course of action that follows; and 3) the identification of factors that impact each portfolio. **Note**: AVOID all references to model names, algorithms, and unnecessary details. Instead, focus on the investment decision.	

© 2022 - WorldQuant University – All rights reserved.